指定ライブラリのみを Databricks で利用する設定手順

Table of Contents

はじめに
- この手順でできること
- 全体の流れ
- 事前に確認すること

手順1：Git フォルダを作る
- 1-1. AWS で CodeCommit 用のパスワードを発行する
- 1-2. Databricks にパスワードを登録する
- 1-3. Git フォルダを作る

手順2：Volume を作る

手順3：wheel を作って Volume に置く
- 3-1. ノートブックを作る
- 3-2. コードを貼り付けて実行する

手順4：ベース環境を作る（Serverless）
- 4-1. 環境定義ファイル（YAML）を作る
- 4-2. ベース環境を作る
- 4-3. ライブラリを更新・追加したとき
- 4-4. ノートブック・ジョブで使う

手順5：外部への通信を遮断する（Serverless）
- 5-1. ネットワークポリシーを作る
- 5-2. ワークスペースに割り当てる
- 5-3. Git フォルダのプルができなくなった場合

手順6：動作を確認する（Serverless）

Classic コンピュートの場合
- 手順C-1：前提の確認（顧客管理 VPC）
- 手順C-2：外部への通信を遮断する（AWS 側）
- 手順C-3：コンピュートポリシーを作る
- 手順C-4：ポリシー以外でクラスタを作れないようにする
- 手順C-5：動作を確認する

留意事項

参考ドキュメント

はじめに

この手順でできること

- Git　フォルダで連携している「指定ライブラリ」を、ノートブックとジョブで使えるようにします（Serverless・Classic の両方）
- それ以外のライブラリ（PyPI や GitHub にあるもの）は、インストールできないようにします

全体の流れ

```
CodeCommit（指定ライブラリ）
   │ 手順1：Git フォルダで連携
   ▼
Databricks Git フォルダ
   │ 手順2・3：wheel を作って Volume に置く
   ▼
Unity Catalog Volume
   ├─ Serverless：手順4 ベース環境に登録
   └─ Classic　：手順C-3 コンピュートポリシーに登録
   ▼
ノートブック・ジョブで使える
```

```
PyPI・GitHub など ── 遮断（Serverless：手順5 ネットワークポリシー／Classic：手順C-2 VPC の出口制御）
```

手順	やること	対象	操作する画面	必要な権限
1	Git フォルダを作る	共通	Databricks ワークスペース	ワークスペースのユーザー
2	Volume を作る	共通	Databricks ワークスペース（カタログ）	カタログの管理者
3	wheel を作って Volume に置く	共通	Databricks ワークスペース（ノートブック）	ワークスペースのユーザー
4	ベース環境を作る	Serverless	Databricks ワークスペース（設定）	ワークスペース管理者
5	外部への通信を遮断する	Serverless	Databricks アカウントコンソール	アカウント管理者
6	動作を確認する	Serverless	Databricks ワークスペース（ノートブック）	ワークスペースのユーザー

C-1	前提の確認（顧客管理 VPC）	Classic	AWS / アカウントコンソール	アカウント管理者
C-2	外部への通信を遮断する	Classic	AWS マネジメントコンソール	AWS のネットワーク管理者
C-3	コンピュートポリシーを作る	Classic	Databricks ワークスペース（コンピュート）	ワークスペース管理者
C-4	ポリシー以外でクラスタを作れないようにする	Classic	Databricks ワークスペース（設定）	ワークスペース管理者
C-5	動作を確認する	Classic	Databricks ワークスペース	ワークスペースのユーザー

Serverless だけを使う場合は手順 1〜6、Classic も使う場合は手順 1〜3 と C-1〜C-5 を実施します（両方使う場合はすべて）。

事前に確認すること

- ワークスペースで Unity Catalog が使えること
- 指定ライブラリの CodeCommit リポジトリに `pyproject.toml`（または `setup.py`）があること
- Serverless：Databricks のプランが **Enterprise** であること（手順5に必要）
- Classic：ワークスペースが 顧客管理 **VPC** で作られていること（手順C-1 参照）

この手順書では、以下の名前を例として使います。お客様の環境に合わせて読み替えてください。

項目	例
Git フォルダの場所	/Workspace/approved-libs/<リポジトリ名>
カタログ	<カタログ>
Volume	<カタログ>.lib_mgmt.approved_wheels
ベース環境の名前	承認済みライブラリ環境
ネットワークポリシーの名前	approved-libs-only

手順**1**：**Git** フォルダを作る

1-1. AWS で **CodeCommit** 用のパスワードを発行する

1）AWS マネジメントコンソールで「IAM」を開く

2） 左メニューの「ユーザー」をクリックし、Databricks から CodeCommit に接続するユーザーをクリック

3）「セキュリティ認証情報」タブをクリック

4）「AWS CodeCommit の HTTPS Git 認証情報」の「認証情報を生成」をクリック

5）表示された ユーザー名 と パスワード をメモする（パスワードはこの画面でしか表示されません）

1-2. Databricks にパスワードを登録する

1）Databricks のワークスペースを開く

2）画面右上の 自分のアイコン をクリック →「設定」をクリック

3）左メニューの「ユーザー」の下にある「リンクされたアカウント」をクリック

4）「Git 統合」の欄で、Git プロバイダーに「**AWS CodeCommit**」を選ぶ

5）1-1 でメモした ユーザー名 と パスワード を入力して「保存」をクリック

1-3. Git フォルダを作る

1）左のサイドバーで「ワークスペース」をクリック

2）一番上の「Workspace」をクリックし、右上の「作成」→「フォルダ」をクリック

3）名前に `approved-libs` と入力して「作成」をクリック

4）作った `approved-libs` フォルダを開き、右上の「作成」→「**Git** フォルダー」をクリック

5）「Git フォルダーの作成」画面で次のように入力する

項目	入力する内容
Git リポジトリの URL	https://git-codecommit.<リージョン>.amazonaws.com/v1/repos/<リポジトリ名>
Git プロバイダー	AWS CodeCommit
Git フォルダ名	自動で入る（そのままでよい）
スパースチェックアウトモード	チェックしない

6）「**Git** フォルダを作成」をクリック

7）フォルダの中に、ライブラリのファイル（`pyproject.toml` など）が表示されれば完了

> ライブラリが更新されたら：Git フォルダの名前の横にある「**Git**」をクリック →「プル」をクリックすると、最新の内容が取り込まれます。

手順2：Volume を作る

wheel ファイル（ライブラリをインストールできる形にしたファイル）を置く場所を作ります。

1）左のサイドバーで「カタログ」をクリック

2）使うカタログ（例：`<カタログ>`）をクリック

3）右上の「作成」→「スキーマ」をクリックし、名前に `lib_mgmt` と入力して「作成」をクリック

4）作った `lib_mgmt` スキーマをクリック

5）右上の「作成」→「ボリューム」をクリック

6）名前に `approved_wheels` と入力し、種類は「マネージド」のまま「作成」をクリック

7）作った `approved_wheels` ボリュームの「権限」タブをクリック →「付与」をクリック

8）プリンシパルに `account users` を選び、権限の「**READ VOLUME**」にチェックを入れて「付与」をクリック

（同じ手順で、`lib_mgmt` スキーマの権限に `account users` の「**USE SCHEMA**」も付与する）

手順3：wheel を作って Volume に置く

3-1. ノートブックを作る

1）左のサイドバーで「ワークスペース」をクリックし、`approved-libs` フォルダを開く

2）右上の「作成」→「ノートブック」をクリック

3）左上のノートブック名をクリックし、`build_wheel` に変更

4）右上の「接続」をクリック →「サーバーレス」を選ぶ

3-2. コードを貼り付けて実行する

1）最初のセルに、次のコードをそのまま貼り付ける

2）1〜2 行目の `<Git フォルダ名>` と `<カタログ>` を、お客様の環境の名前に書き換える

```
src  = "/Workspace/approved-libs/<Git フォルダ名>"
dest = "/Volumes/<カタログ>/lib_mgmt/approved_wheels"
```

```
import glob, os, shutil, subprocess, sys, tempfile
work = tempfile.mkdtemp()
pkg = os.path.join(work, "pkg")
```

```
shutil.copytree(src, pkg, ignore=shutil.ignore_patterns(".git*", "dist",
   "build", "*.egg-info"))
out = os.path.join(work, "dist")
subprocess.run([sys.executable, "-m", "pip", "wheel", "--no-deps",
   "--no-build-isolation",
                "-w", out, pkg], check=True)
for whl in glob.glob(f"{out}/*.whl"):
    shutil.copy(whl, dest)
    print("配置しました:", os.path.join(dest, os.path.basename(whl)))
```

3) セルの左にある ▸（実行）をクリック

4) 最後に次のような行が表示されれば成功。このパスをメモする（手順4で使います）

```
配置しました: /Volumes/<カタログ>/lib_mgmt/approved_wheels/<ライブラリ名
   >-0.1.0-py3-none-any.whl
```

> ライブラリが更新されたら：手順1-3 の「プル」をしてから、このノートブックをもう一度実行します。

手順4：ベース環境を作る（Serverless）

「新しいノートブックを開くと、指定ライブラリが最初から入っている」状態にします。

4-1. 環境定義ファイル（YAML）を作る

1) 左のサイドバーで「ワークスペース」をクリックし、`approved-libs` フォルダを開く

2) 右上の「作成」→「ファイル」をクリック

3) ファイル名に `approved-base-env.yaml` と入力して作成

4) 次の内容を貼り付け、3 行目を手順3でメモしたパスに書き換える（入力内容は自動で保存されます）

```
environment_version: "4"
dependencies:
  - /Volumes/<カタログ>/lib_mgmt/approved_wheels/<ライブラリ名
   >-0.1.0-py3-none-any.whl
```

> ライブラリを増やすときは、`dependencies` の下に `  - /Volumes/...whl` の行を追加します。

4-2. ベース環境を作る

1）画面右上の 自分のアイコン をクリック →「設定」をクリック

2）左メニューの「ワークスペース管理」の下にある「コンピュート」をクリック

3）「サーバレスコンピュートの基本環境」の横にある「管理」をクリック

4）「作成」をクリック

5）次のように入力する

項目	入力する内容
名前	承認済みライブラリ環境（利用者の画面にこの名前が表示されます）
環境仕様ファイル	ファイル選択画面で /Workspace/approved-libs/approved-base-env.yaml を選ぶ
すべての新しいノートブックのデフォルトにする	オンにする

6）「作成」をクリック

7）一覧の「ステータス」が「すぐに使用できます。」になるまで待つ（数分かかります）

8）作った環境の横の 星のアイコン が付いていることを確認する（付いていなければクリックする）

4-3. ライブラリを更新・追加したとき

1）4-1 の YAML ファイルを開き、wheel のパスを書き換える（または行を追加する）

2）4-2 の画面（設定 → コンピュート →「サーバレスコンピュートの基本環境」の「管理」）を開く

3）対象の環境の右端にある ⋮（ケバブメニュー）をクリック →「更新」をクリック

4）「確認」をクリック

5）更新後に 新しく開いた ノートブックから反映されます（開いているノートブックは、接続し直すと反映されます）

4-4. ノートブック・ジョブで使う

場面	操作
新しいノートブック	何もしなくてよい（自動で 承認済みライブラリ環境 になります）
以前からあるノートブック	右側の「環境」アイコンをクリック →「基本環境」で 承認済みライブラリ環境 を選ぶ →「適用」をクリック

ジョブ	タスクの設定で「環境」の編集を開き、基本環境に 承認済みライブラリ環境 を選ぶ（選択肢に出ない場合は、YAML ファイルのパス /Workspace/approved-libs/approved-base-env.yaml を指定する）

手順5：外部への通信を遮断する（Serverless）

この設定は アカウントコンソール で行います（ワークスペースの画面とは別です）。

5-1. ネットワークポリシーを作る

1）ブラウザで https://accounts.cloud.databricks.com を開き、アカウント管理者でログイン

2）左メニューの「セキュリティ」をクリック

3）「ネットワーク」タブをクリック

4）「ポリシー」の下にある「コンテキストベースのイングレスおよびエグレス制御」をクリック

5）「新しいネットワーク ポリシーの作成」をクリック

6）「ポリシー名」に `approved-libs-only` と入力

7）「出力」タブをクリック

8）「特定の宛先へのアクセス制限」を選ぶ

9）許可する宛先（ドメイン）は 何も追加しない

- これで PyPI・GitHub など、インターネット上のすべての宛先への通信が遮断されます
- Volume（Unity Catalog のストレージ）へのアクセスは自動で許可されるので、追加は不要です

10）「ポリシーの施行」のドライランは 選ばない（選ぶと遮断されず、記録だけになります）

11）保存する

> 影響を先に確かめたい場合：手順10 で「すべての製品」のドライランを選ぶと、通信は遮断されずに記録だけされます。記録は手順6-3 のクエリで確認でき、`access_type` が `DRY_RUN_DENIAL` になります。問題がなければドライランを外して保存し直します。

5-2. ワークスペースに割り当てる

1）アカウントコンソールの左メニューで「ワークスペース」をクリック

2）対象のワークスペースをクリック

3）「ネットワーク ポリシー」の欄で「ネットワーク ポリシーの更新」をクリック

4）一覧から `approved-libs-only` を選ぶ

5）「ポリシーの適用」をクリック

6）**10** 分ほど待つ（すぐには反映されません）

5-3. Git フォルダのプルができなくなった場合

ワークスペースの 設定 →「プレビュー」で「**Git** フォルダーの **Git CLI** サポート」がオンになっていると、Git の操作もこの遮断の対象になり、プルが失敗します。次のどちらかで対応してください。

- 「Git フォルダーの Git CLI サポート」をオフにして、Git フォルダを作り直す
- 5-1 の許可する宛先に `git-codecommit.<リージョン>.amazonaws.com` を追加する

手順**6**：動作を確認する（**Serverless**）

1）左のサイドバーで「ワークスペース」→ 任意のフォルダで「作成」→「ノートブック」をクリック

2）右上の「接続」→「サーバーレス」を選ぶ

3）次の 3 つのセルを順に実行し、結果が表のとおりになることを確認する

セル**1**：指定ライブラリが使える

```
import <ライブラリのモジュール名>
print(<ライブラリのモジュール名>.__version__)
```

セル**2**：**PyPI** からはインストールできない

```
%pip install emoji --timeout 10 --retries 0
```

セル**3**：**GitHub** からはインストールできない

```
%pip install git+https://github.com/carpedm20/emoji.git --timeout 10
   --retries 0
```

セル	期待する結果
1	バージョン番号が表示される
2	エラーになる（No matching distribution found for emoji）
3	エラーになる（git clone に失敗した旨のメッセージ）

6-3. 遮断された通信の記録を見る（任意）

左のサイドバーの「**SQL**　エディタ」で次を実行すると、遮断された通信の一覧が見られます（記録されるまで時間がかかることがあります）。

```
SELECT event_time, destination, access_type
FROM system.access.outbound_network
WHERE event_time >= current_timestamp() - INTERVAL 1 DAY
ORDER BY event_time DESC;
```

Classic コンピュートの場合

手順 1〜3（Git フォルダ・Volume・wheel）は Serverless と共通です。そのうえで、以下を実施します。

> Classic の手順は公式ドキュメントに基づくものです。Classic 環境での動作確認は、導入時にお客様環境で実施してください。

手順**C-1**：前提の確認（顧客管理 **VPC**）

Classic コンピュートの外部通信を遮断するには、ワークスペースが 顧客管理 **VPC**（お客様の AWS アカウントの VPC）で作られている必要があります。

1) アカウントコンソール（https://accounts.cloud.databricks.com）を開く

2) 左メニューの「ワークスペース」をクリックし、対象のワークスペースをクリック

3) ネットワーク設定の欄で、お客様の VPC（ネットワーク設定）が指定されていることを確認する

確認結果	対応
お客様の VPC が指定されている	手順C-2 に進む
指定されていない（Databricks 管理の VPC）	既存のワークスペースを顧客管理 VPC に移すことはできません。顧客管理 VPC で新しいワークスペースを作る必要があります

手順**C-2**：外部への通信を遮断する（**AWS** 側）

この手順は AWS 側の作業です。ワークスペースの VPC から外に出る通信を、ファイアウォール（AWS Network Firewall やプロキシ）で制御します。

1) ワークスペースのサブネットのルートテーブルで、`0.0.0.0/0` の宛先をファイアウォール（またはプロキシ）経由にする

2) ファイアウォールの許可リストに、**Databricks** の動作に必要な通信先だけ を登録する（下の表）

3) PyPI（`pypi.org`、`files.pythonhosted.org`）や GitHub（`github.com` など）は 許可リストに入れない（それ以外もすべて拒否）

4） S3・STS・Kinesis は、VPC エンドポイントを作って AWS 内の経路で通すことを推奨（ファイアウォールを通らなくなります）

通信先	必要性	備考
Databricks の Web アプリ（REST API を含む）	必須	リージョンごとのドメインは「IP アドレスとドメイン」のページで確認
セキュアクラスター接続（SCC リレー）	必須（セキュアクラスター接続を使う場合）	同上
コントロールプレーンのインフラ	必須	同上
s3.amazonaws.com:443	必須	ワークスペースのルート S3 バケット
s3.<リージョン>.amazonaws.com:443	必要に応じて	VPC エンドポイント推奨
sts.amazonaws.com:443、STS のリージョン URL	必須	VPC エンドポイント推奨
Kinesis のリージョン URL	必須	ログ送信に使用
RDS（レガシー Hive メタストア）	使う場合のみ	Unity Catalog だけなら不要

- 通信先は、IP アドレスではなくドメイン名 で指定することが推奨されています
- Volume（Unity Catalog のストレージ = S3）へのアクセスは、上の S3 の許可で通ります
- 設定後、クラスタが起動しない場合は、許可リストの不足を疑ってください

手順**C-3**：コンピュートポリシーを作る

クラスタの起動時に、指定ライブラリが自動でインストールされるようにします。

1） 左のサイドバーで「コンピュート」をクリック

2）「ポリシー」タブをクリック

3）「ポリシーの作成」をクリック

4） 名前に `approved-libs-policy` と入力

5）「説明」に `指定ライブラリを自動インストールするポリシー` などと入力

6）「定義」タブで「定義を **JSON** として編集」をクリックし、次の内容を貼り付ける

```
{
  "data_security_mode": {"type": "fixed", "value": "USER_ISOLATION"},
  "autotermination_minutes": {"type": "range", "maxValue": 120,
"defaultValue": 60}
}
```

- 1 行目：アクセスモードを「標準」に固定します
- 2 行目：使っていないクラスタを自動で停止します（任意）

7）画面下部の「ライブラリ」タブをクリック

8）「ライブラリの追加」をクリック

9）ライブラリソースで「ボリューム」を選び、手順3 で作った wheel ファイル（`/Volumes/<カタログ>/lib_mgmt/approved_wheels/<ライブラリ名>-0.1.0-py3-none-any.whl`）を選ぶ

10）「追加」をクリック（ライブラリが複数あるときは 8〜10 を繰り返す）

11）「作成」をクリック

利用者に使用権限を付ける

1）作ったポリシー `approved-libs-policy` をクリック

2）「すべての権限を表示」をクリック

3）利用者（またはグループ）を選び、権限に「使用可能」を選んで追加・保存

> ライブラリを更新・追加したとき：ポリシーを開いて「編集」→「ライブラリ」タブで wheel を差し替え（または追加）→ 保存します。反映は、クラスタを 再起動 した後です。

手順**C-4**：ポリシー以外でクラスタを作れないようにする

利用者が、ポリシーを使わずに自由な設定でクラスタを作れないようにします。

1）画面右上の 自分のアイコン をクリック →「設定」をクリック

2）「**ID** とアクセス」をクリック

3）「グループ」の横にある「管理」をクリック（ユーザー単位の場合は「ユーザー」の横の「管理」）

4）対象のグループ（またはユーザー）をクリック

5）「無制限のクラスター作成を許可する」のトグルを オフ にする

6）「コンピュート」→「ポリシー」タブで、既定のポリシー（「Personal Compute」「Power User Compute」など）の「すべての権限を表示」を開き、不要な利用者の権限を外す

- `users` グループ（全ユーザー）で変更すると、全員に影響します
- グループから引き継いだ権限は、トグルがグレー表示になり個別には外せません。元のグループ側で外してください

手順**C-5**：動作を確認する

1）左のサイドバーで「コンピュート」をクリック →「コンピュートを作成」をクリック

2）「ポリシー」で `approved-libs-policy` を選ぶ

3）名前を入力して「コンピュートを作成」をクリックし、起動するまで待つ

4） 起動したクラスタの「ライブラリ」タブで、指定ライブラリの状態が「インストール済み」になっていることを確認する

5）新しいノートブックを作り、右上の「接続」でこのクラスタを選ぶ

6）手順6 の 3 つのセルを実行し、同じ結果になることを確認する（セル1 は成功、セル2・3 は失敗）

———

留意事項

- 依存パッケージがあるライブラリ：外部から取得できないため、依存パッケージの wheel も Volume に置き、YAML の `dependencies` に追加してください
- 実行時に外部からデータを取りに行くライブラリ（Hugging Face のモデル、nltk のデータなど）：通信が遮断されるため、実行時にエラーになります
- **Databricks** に最初から入っているライブラリ（pandas、numpy など）：そのまま使えます
- **Classic** で既にインストール済みのライブラリ：コンピュートポリシーのライブラリは、クラスタの起動（再起動）時にインストールされます

———

参考ドキュメント

- Git プロバイダーとの連携（認証情報の登録）：https://docs.databricks.com/aws/ja/repos/get-access-tokens-from-git-provider
- Git フォルダの操作（作成・プル）：https://docs.databricks.com/aws/ja/repos/git-operations-with-repos
- Unity Catalog Volume：https://docs.databricks.com/aws/ja/volumes/
- ワークスペースのベース環境：https://docs.databricks.com/aws/ja/admin/workspace-settings/base-environment
- Serverless の環境設定：https://docs.databricks.com/aws/ja/compute/serverless/dependencies
- Serverless の外部通信制御：https://docs.databricks.com/aws/ja/security/network/serverless-network-security/network-policies
- ネットワークポリシーの管理：https://docs.databricks.com/aws/ja/security/network/serverless-network-security/manage-network-policies
- 外部通信の記録（システムテーブル）：https://docs.databricks.com/aws/ja/admin/system-tables/network
- 顧客管理 VPC（Classic）：https://docs.databricks.com/aws/ja/security/network/classic/customer-managed-vpc
- IP アドレスとドメイン（Classic の許可リスト）：https://docs.databricks.com/aws/ja/resources/ip-domain-region

- コンピュートポリシーの作成とライブラリ（Classic）:
  https://docs.databricks.com/aws/ja/admin/clusters/policies
- コンピュートポリシーの定義（Classic）:
  https://docs.databricks.com/aws/ja/admin/clusters/policy-definition
- エンタイトルメント（無制限のクラスター作成）:
  https://docs.databricks.com/aws/ja/security/auth/entitlements